Mini Project Report On

SchemeSmart-AI: AI powered Government Scheme Recommendation System

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

By

Anna Maria Jose (U2303048)

Anna Rose Shaji (U2303049)

Arunima V K (U2303057)

Ayisha Noureen (U2303066)

Under the guidance of

Mr Tonny Binoy

Department of Computer Science & Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Affiliated to APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

May 2024

CERTIFICATE

This is to certify that the mini project report entitled ***"SchemeSmart AI : AI-powered Government Scheme Recommendation System"*** *is a bonafide record of the work done by* ***Anna Maria Jose (U2303048)***, ***Anna Rose Shaji (U2303049)***, ***Arunima V K (U2303057)***, ***Ayisha Noureen (U2303066)***, *submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2023-2024.*

Mr.Tonny Binoy
Assistant Professor
Dept. of CSE
RSET

Dr.Mary Priya Sebastian
Associate Professor
Dept. of CSE
RSET

Dr.Preetha K.G.
Professor, Head of the Department
Dept. of CSE
RSET

ACKNOWLEDGEMENTS

I wish to express my sincere gratitude towards Dr Jaison Paul Mulerikkal CMI, Principal of RSET, and Dr. Preetha K.G., Head of the Department of Computer Science and Engineering for providing me with the opportunity to undertake my mini project, "chemeSmart AI : AI- powered Government Scheme Recommendation System".

I am highly indebted to my project coordinators, **Dr.Mary Priya Sebastian**, Associate Professor, Department of Computer Science and Engineering for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Mr.Tonny Binoy** for his/her patience and all the priceless advice and wisdom he/she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Anna Maria Jose

Anna Rose Shaji

Arunima V K

Ayisha Noureen

Abstract

SchemeSmart AI is a web-based platform designed to simplify access to government welfare schemes by providing personalized recommendations and guided application support. Many citizens face challenges in identifying suitable schemes due to scattered information, complex eligibility criteria, and lack of awareness. This project addresses these issues by integrating an intelligent recommendation system with a user-friendly interface that analyzes user profiles and suggests relevant schemes efficiently. The system utilizes a FastAPI-based backend and a React frontend to deliver real-time recommendations, interactive form guidance, and chatbot assistance. Unlike existing platforms that provide static and generic information, SchemeSmart AI offers dynamic, user-specific suggestions and step-by-step form-filling support, reducing errors and improving accessibility. Additionally, the platform enhances user engagement through features such as notifications, scheme details, and simplified navigation. The key advantages of this project include improved accessibility to welfare schemes, reduced information overload, personalized recommendations, and enhanced user experience through automation and interactivity. This makes the system particularly beneficial for individuals with limited digital literacy.

Contents

Acknowledgements **i**

Abstract **ii**

List of Figures **vi**

List of Tables **vii**

List of Abbreviations **viii**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 2

1.3 Scope and Motivation 2

1.4 Objectives 3

1.5 Challenges 3

1.6 Assumptions 3

1.7 Societal / Industrial Relevance 4

1.8 Organization of the Report 4

2 Software Requirements Specification **6**

2.1 Introduction 6

2.2 Overall Description 6

2.3 External Interface Requirements 7

2.4 System Features 7

2.5 Other Nonfunctional Requirements 8

3 System Architecture and Design **9**

3.1 System Overview 9

3.2 Architectural Design 10

3.3 Dataset identified 11
3.4 Proposed Methodology/Algorithms 13
3.5 User Interface Design 16
3.6 Database Design 18
3.7 Description of Implementation Strategies 21
3.7.1 Backend Implementation using FastAPI 21
3.7.2 Database and Authentication using Supabase 21
3.7.3 Recommendation Engine Implementation 22
3.7.4 Chatbot Implementation using TF-IDF and RAG 22
3.7.5 Guided Form-Filling Implementation 23
3.7.6 Frontend Implementation using React 23
3.7.7 Authentication Strategy 24
3.7.8 Evaluation Strategies 24
3.7.9 Summary 24
3.8 Module Division 24
3.8.1 Module 1: User Profile Management 24
3.8.2 Module 2: Data Acquisition & Dataset Preparation 25
3.8.3 Module 3: Eligibility Filtering Engine 25
3.8.4 Module 4: AI-Based Scheme Ranking Module 25
3.8.5 Module 5: Scheme Recommendation Interface 25
3.8.6 Module 6: Application Form Template Module 25
3.8.7 Module 7: AI Voice-Assisted Form Guidance 25
3.8.8 Module 8: AI Chatbot Support Module 26
3.8.9 Module 9: Dashboard & Notification System 26
3.8.10 Module 10: Backend & System Integration 26
3.8.11 Module Allocation to Team Members 26
3.9 Work Schedule - Gantt Chart 27

4 Results and Discussions **28**
4.1 Overview 28
4.2 Testing 29

5 Conclusion **30**

5.1 Conclusion 30

5.2 Future Scope 30

Appendix A: Presentation **34**

List of Figures

3.1 System Architecture 11

3.2 Landing Page Interface Showing Navigation and Introduction to SchemeSmart AI 16

3.3 User Registration and Login Interface 16

3.4 User Profile Creation Interface for Collecting Demographic Details 17

3.5 Interface Displaying Personalized Scheme Recommendations 17

3.6 Scheme Detail Page Showing Eligibility, Benefits, and Required Documents 17

3.7 Guided Form-Filling Interface for Scheme Application 17

3.8 Chatbot Interface for Query Handling and User Assistance 18

3.9 Gantt Chart Representing Project Timeline 27

List of Tables

3.1 User table schema (Supabase) 19
3.2 Scheme data table structure 19
3.3 User profile table schema 20
3.4 Notification system table schema 20

List of Abbreviations

AI - Artificial Intelligence

API - Application Programming Interface

NLP - Natural Language Processing

TF-IDF - Term Frequency–Inverse Document Frequency

RAG - Retrieval-Augmented Generation

UI - User Interface

JSON - JavaScript Object Notation

HTTP - Hypertext Transfer Protocol

HTTPS - Hypertext Transfer Protocol Secure

REST - Representational State Transfer

BaaS - Backend as a Service

DB - Database

Chapter 1

Introduction

1.1 Background

In recent years, the Government of India has introduced numerous welfare schemes aimed at improving the socio-economic conditions of its citizens across sectors such as agriculture, healthcare, education, housing, and employment. Despite the availability of these schemes, a significant portion of the population remains unaware of their existence or faces difficulties in accessing them. This gap is primarily due to fragmented information sources, complex eligibility criteria, and the lack of a centralized, user-friendly platform.

Currently, most government portals provide static information, requiring users to manually search, interpret eligibility conditions, and navigate lengthy application procedures. This process can be overwhelming, especially for individuals with limited digital literacy or those residing in rural areas. Additionally, users often struggle to determine which schemes they are eligible for, leading to missed opportunities and underutilization of government resources.

With the rapid advancement of web technologies and intelligent systems, there is an increasing need for platforms that can simplify access to such information through personalization and automation. Integrating technologies like recommendation systems, interactive interfaces, and chatbot assistance can significantly improve user experience by delivering relevant information tailored to individual needs.

The SchemeSmart AI project is developed in response to these challenges. It aims to bridge the gap between citizens and government welfare schemes by providing a centralized platform that offers personalized scheme recommendations and guided application

support. By leveraging modern web frameworks and intelligent processing, the system reduces complexity, enhances accessibility, and ensures that users can easily discover and apply for schemes suited to their profiles.

This project is important as it not only improves awareness and accessibility but also contributes to better utilization of government initiatives, ultimately supporting inclusive growth and digital empowerment.

1.2 Problem Definition

The aim of this project is to develop an intelligent platform that simplifies the discovery and application process of government welfare schemes through personalized recommendations and guided support.

Currently, users face difficulty in identifying relevant schemes due to scattered information, complex eligibility criteria, and lack of personalized assistance.

1.3 Scope and Motivation

The scope of the SchemeSmart AI project includes the design and development of a web-based platform that provides personalized recommendations for government welfare schemes. The system focuses on collecting user profile data and matching it with scheme eligibility criteria to deliver accurate and relevant suggestions. It also includes features such as detailed scheme information, guided form-filling assistance, and chatbot-based support to enhance user interaction. The platform is built using modern technologies like FastAPI and React to ensure scalability, responsiveness, and real-time performance. While the current implementation targets general welfare schemes, the system can be further expanded to include region-specific schemes, multilingual support, and mobile application integration.

The motivation behind this project arises from the challenges faced by citizens in accessing and understanding government welfare schemes. Many individuals, especially in rural and underprivileged communities, lack awareness or find it difficult to navigate complex application processes. Existing systems often provide generic information with-

out personalization, making it hard for users to identify schemes relevant to them. This inspired the development of a solution that simplifies the process through automation, personalization, and an intuitive interface. The project aims to bridge the gap between government initiatives and citizens, ensuring that the benefits of welfare schemes reach the intended audience effectively.

1.4 Objectives

1. To develop a user-friendly web platform for accessing information about government welfare schemes.
2. To implement a personalized recommendation system that suggests relevant schemes based on user profiles.
3. To simplify the scheme application process through guided form-filling assistance.
4. To provide an interactive chatbot for resolving user queries and improving user engagement.
5. To ensure efficient data handling and real-time responses using a robust backend framework.
6. To enhance accessibility and awareness of government schemes, especially among users with limited digital literacy.

1.5 Challenges

One of the major challenges in this project is accurately mapping user data with diverse and complex eligibility criteria of multiple government schemes. Ensuring a seamless and responsive user experience while handling real-time recommendations and interactions is also challenging. Additionally, managing incomplete or inconsistent user inputs and presenting clear, understandable guidance to users adds to the complexity of the system.

1.6 Assumptions

1. It is assumed that users provide accurate and complete information in their profiles for effective scheme recommendations.

2. The system assumes that the scheme data used is up-to-date and correctly reflects eligibility criteria and benefits.
3. It is assumed that users have basic internet access and minimal digital literacy to interact with the platform.

1.7 Societal / Industrial Relevance

The SchemeSmart AI project holds significant societal relevance by improving access to government welfare schemes for citizens, particularly those from rural and underprivileged communities. By providing personalized recommendations and simplifying the application process, the platform helps bridge the gap between government initiatives and the intended beneficiaries. This leads to increased awareness, better utilization of public resources, and supports inclusive growth by ensuring that eligible individuals do not miss out on available benefits.

From an industrial perspective, the project demonstrates the effective use of modern web technologies and intelligent systems in solving real-world problems. It can be adopted by government agencies, NGOs, and digital service providers to enhance citizen engagement and streamline service delivery. Additionally, the system can be scaled and customized for other domains such as healthcare, education, and financial services, making it a versatile solution for digital transformation initiatives.

1.8 Organization of the Report

This report is structured into three main chapters to provide a systematic understanding of the project. Chapter 1 presents the introduction, which includes the background, problem definition, scope and motivation, objectives, challenges, assumptions, and the overall relevance of the project.

Chapter 2 focuses on the Software Requirements Specification (SRS), detailing the overall system description, external interface requirements, system features, and non-functional requirements. This chapter provides a clear understanding of what the system is expected to achieve and the constraints involved.

Chapter 3 describes the System Architecture and Design, including the system overview, architectural design, dataset used, proposed methodology, user interface design, database design, implementation strategies, module division, and project schedule. This chapter explains how the system is structured and implemented to meet the defined requirements.

Chapter 2

Software Requirements Specification

Insert your SRS document here.

2.1 Introduction

SchemeSmart AI is an intelligent web-based system designed to assist citizens in identifying suitable government schemes based on their personal profile. Many individuals are often unaware of the various government benefits available to them or find it difficult to understand complex eligibility criteria. This system addresses these challenges by integrating machine learning techniques with modern web technologies to provide personalized recommendations. The application collects user information such as age, income, occupation, and category, and processes this data to suggest relevant schemes using an AI-based ranking mechanism. Additionally, it incorporates a chatbot powered by a large language model that allows users to ask questions and receive clear, human-like responses. The system is developed using React and Vite for the frontend, FastAPI for the backend, TF-IDF or semantic similarity methods for ranking, Llama-3 for chatbot functionality, and Supabase as the database. The primary objective of the system is to simplify access to government schemes and enhance user awareness and usability.

2.2 Overall Description

SchemeSmart AI is designed as a multi-layered system consisting of four major components: the frontend layer, backend layer, AI engine, and data layer. The frontend layer provides an interactive user interface through which users can enter their profile details, view scheme recommendations, and interact with the chatbot. The backend layer acts as an intermediary that processes user requests, handles authentication, and manages communication between the frontend, AI engine, and database. The AI engine performs the

core intelligence of the system by ranking government schemes based on user profiles using TF-IDF or semantic similarity techniques, and by generating chatbot responses using a large language model. The data layer stores structured information, including government scheme details in JSON format and user data in a Supabase PostgreSQL database. The system follows a client-server architecture in which the frontend communicates with backend APIs, and the backend interacts with both the AI engine and the database to process and return results efficiently.

2.3 External Interface Requirements

The system provides a web-based user interface that can be accessed through standard web browsers on various devices such as laptops, desktops, and mobile phones. The interface includes forms for entering user details, a dashboard for displaying recommended schemes along with match scores, and a chatbot interface for handling user queries. From a hardware perspective, the system does not require any specialized equipment and can run on any device with internet connectivity. In terms of software interfaces, the system integrates multiple technologies including React and Vite for the frontend, FastAPI for the backend, Supabase for database management, and AI models such as TF-IDF and Llama-3 for intelligent processing. Communication between components is achieved through HTTP/HTTPS protocols using RESTful APIs. The system also incorporates mathematical models for similarity calculations, such as cosine similarity and TF-IDF scoring, which are used to determine the relevance of schemes based on user input.

2.4 System Features

The SchemeSmart AI system includes several important features that enhance its functionality and user experience. It provides a secure user registration and login mechanism that allows users to create accounts and store their profile information safely. Based on the user's input, the system performs profile-based recommendations by analyzing key attributes such as age, income, and category, and suggesting the most relevant government schemes. The recommendation process is powered by an AI-based ranking mechanism that uses TF-IDF or semantic similarity techniques to improve accuracy. The system also includes a chatbot feature that enables users to ask questions about schemes and receive

clear, human-like responses generated by a large language model. Additionally, users have the option to bookmark schemes for future reference. The system ensures real-time data retrieval so that users receive instant responses and updated information whenever they interact with the platform.

2.5 Other Nonfunctional Requirements

The system is designed to meet several non-functional requirements that ensure its overall quality and reliability. In terms of performance, the system provides fast response times for both recommendations and chatbot interactions by efficiently handling API requests. Scalability is achieved through the use of cloud-based services such as Supabase, allowing the system to support a growing number of users without performance degradation. Security is maintained through secure authentication mechanisms, such as JWT-based authentication, and proper data protection practices. The system emphasizes usability by offering a simple and intuitive user interface that is easy to navigate even for non-technical users. Reliability is ensured by designing the system to function consistently with minimal downtime or errors. Maintainability is supported through a modular architecture, which allows developers to easily update or replace components such as AI models or database structures. Finally, the system ensures high availability, enabling users to access its services at any time through a web browser.

Chapter 3

System Architecture and Design

3.1 System Overview

SchemeSmart AI is a web-based intelligent platform designed to simplify the discovery and application of government welfare schemes through personalization and automation. The system integrates a modern frontend interface with a robust backend to provide real-time recommendations, guided assistance, and interactive support to users. It follows a client-server architecture where the frontend, developed using React, communicates with a FastAPI-based backend through RESTful APIs.

At a high level, the system operates by collecting user-specific information such as age, occupation, income level, education, and location. This data is processed by the backend, where it is matched against a structured dataset of government schemes containing eligibility criteria, benefits, and required documentation. Based on this analysis, the system generates personalized recommendations tailored to the user's profile. The results are then displayed on the frontend in an easy-to-understand format, along with detailed descriptions and actionable steps.

The architecture of the system consists of three major layers: the presentation layer, the application layer, and the data layer. The presentation layer includes the user interface components such as registration, login, profile management, scheme listing, and chatbot interaction. The application layer handles business logic, including authentication, recommendation processing, and API management. The data layer consists of databases that store user information, scheme datasets, and interaction logs.

The workflow of the system begins with user authentication, where new users can reg-

ister and existing users can log in securely. After authentication, users create or update their profile, which is essential for generating accurate recommendations. Once the profile is submitted, the recommendation engine processes the data and filters schemes based on eligibility conditions. The system then ranks and presents the most relevant schemes to the user.
In addition to recommendations, the platform provides a guided form-filling feature that assists users in completing application forms step by step. This reduces errors and improves the overall success rate of applications. The chatbot module further enhances usability by answering user queries, explaining scheme details, and guiding users through different sections of the platform.

The system also includes features such as viewing detailed scheme information, tracking application progress, and receiving notifications about new or relevant schemes. These functionalities ensure continuous user engagement and keep users informed about opportunities they may benefit from.

From a technical perspective, the backend is implemented using FastAPI, which enables high-performance API handling and asynchronous operations. The frontend is built with React, ensuring a responsive and dynamic user experience. Data exchange between the frontend and backend is handled using JSON over HTTP, ensuring compatibility and scalability.

Overall, the SchemeSmart AI system is designed to provide a seamless, efficient, and user-centric solution for accessing government welfare schemes. By integrating intelligent recommendation mechanisms with a structured architecture, the platform effectively bridges the gap between citizens and government services, making the entire process more accessible, transparent, and efficient.

3.2 Architectural Design

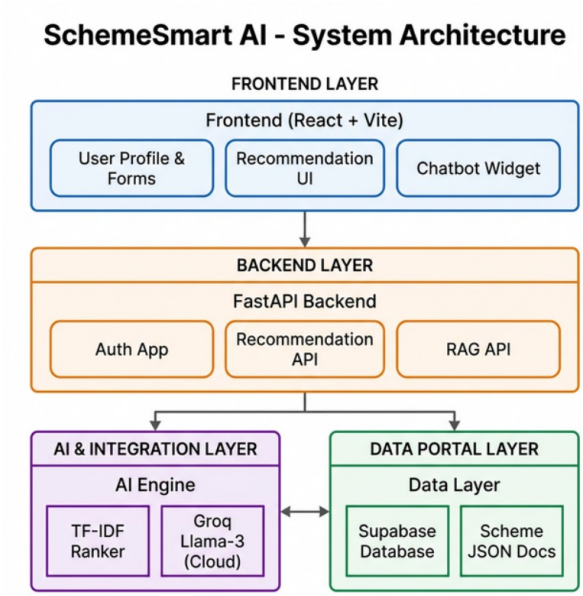


Figure 3.1: System Architecture

3.3 Dataset identified

The dataset used in the SchemeSmart AI project consists of structured information about various government welfare schemes. The data is collected from official government portals and publicly available resources, ensuring authenticity and relevance. Each scheme entry contains important attributes such as scheme ID, scheme name, sector, state, implementing department, description, benefits, eligibility criteria, and required documents.

The dataset is maintained in a structured format (JSON), which allows easy integration with the FastAPI backend and efficient processing for recommendation purposes. This format supports scalability and flexibility, enabling the addition of new schemes or modification of existing data without affecting system performance. The dataset is stored and accessed through the backend, where it is filtered and matched against user profiles to generate personalized recommendations.

Key properties of the dataset include:

- Structured Format: Organized in JSON for easy parsing and retrieval.

• Multi-sector Coverage: Includes schemes from domains such as healthcare, agriculture, education, housing, and employment.

• Eligibility Mapping: Contains detailed eligibility conditions used by the recommendation engine.

• Pan-India and State-Level Data: Covers both central and state government schemes.

A sample subset of the dataset is shown below:


```
{
  "scheme_id": "HEA_SCH_017",
  "scheme_name": "Ayushman Bharat { Pradhan Mantri Jan Arogya Yojana",
  "sector": "Healthcare",
  "state": "All India",
  "implementing_department": "Ministry of Health and Family Welfare",
  "description": "Provides health insurance coverage for secondary and tertiary
  "benefits": [
    "Coverage up to 5 lakh per family per year",
    "Cashless treatment at empanelled hospitals"
  ],
  "eligibility": [
    "Economically weaker sections",
    "Based on SECC database"
  ],
  "documents_required": [
    "Aadhaar Card",
    "Ration Card",
    "Income Certificate"
  ]
}
```

This dataset serves as the core component of the system, enabling accurate matching between user profiles and scheme requirements. It plays a crucial role in ensuring that users receive relevant, reliable, and up-to-date recommendations.

3.4 Proposed Methodology/Algorithms

The SchemeSmart AI system uses a rule-based recommendation methodology combined with structured data filtering to provide personalized government scheme suggestions. The approach primarily focuses on matching user profile attributes with scheme eligibility criteria and ranking the results based on relevance.

The overall methodology consists of three major stages: data collection, eligibility matching, and recommendation generation.

Step 1: User Data Collection

The system collects user-specific information such as:

- Age
- Gender
- Occupation
- Income
- Education
- Location

This data is stored in the user profile and serves as the input for the recommendation process.

Step 2: Scheme Filtering

The backend retrieves all available schemes from the dataset and applies filtering based on:

- Location (State/National schemes)
- Sector preferences
- Basic eligibility constraints

This step reduces the search space and improves efficiency.

Step 3: Eligibility Matching Algorithm

Each scheme's eligibility criteria are compared with the user's profile to determine suitability.

Algorithm 1: Scheme Eligibility Matching

Input: UserProfile U, SchemeDataset S

Output: List of Eligible Schemes E

1. Initialize empty list E
2. For each scheme s in S:
3. Set matchscore = 0
4. For each eligibility condition c in s:
5. If U satisfies condition c:
6. Increment matchscore
7. If matchscore threshold:
8. Add s to E
9. Return E

Step 4: Recommendation Ranking

After identifying eligible schemes, the system ranks them based on relevance.

Algorithm 2: Scheme Ranking

Input: Eligible Schemes E

Output: Ranked Scheme List R

1. For each scheme s in E:
2. Calculate score based on:
- Number of matched conditions
- Priority factors (income, category, etc.)
3. Sort schemes in descending order of score
4. Return sorted list R

Step 5: Recommendation Output

The top-ranked schemes are displayed to the user along with:

- Scheme details
- Benefits
- Required documents
- Application guidance

Step 6: Chatbot Assistance

The chatbot module enhances user interaction by answering queries related to schemes, eligibility, and navigation. It uses a combination of TF-IDF (Term Frequency–Inverse Document Frequency) and Retrieval-Augmented Generation (RAG)This methodology ensures that users receive accurate, relevant, and easy-to-understand recommendations, improving accessibility and usability of government welfare schemes.

Algorithm 3: Chatbot Query Processing

Input: User Query Q
Output: Response R

1. Preprocess query Q (tokenization, stop-word removal)
2. Convert Q into TF-IDF vector

3. Compute similarity between Q and dataset documents
4. Retrieve top-k relevant documents D
5. Pass D along with Q to RAG model
6. Generate contextual response R
7. Return response to user

3.5 User Interface Design

Figure 3.2: Landing Page Interface Showing Navigation and Introduction to SchemeSmart AI

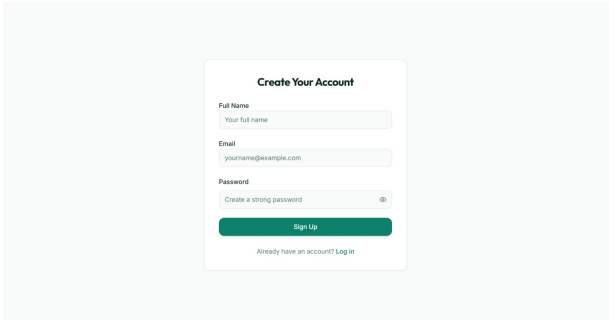


Figure 3.3: User Registration and Login Interface

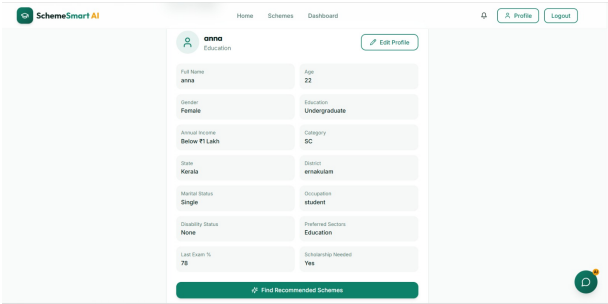


Figure 3.4: User Profile Creation Interface for Collecting Demographic Details

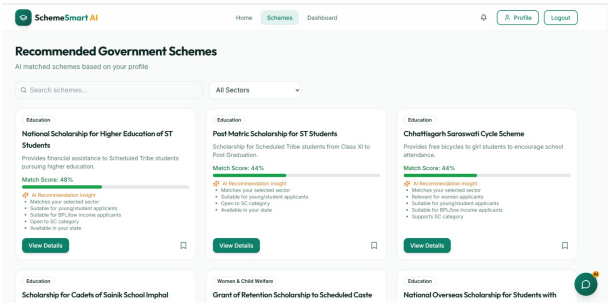


Figure 3.5: Interface Displaying Personalized Scheme Recommendations

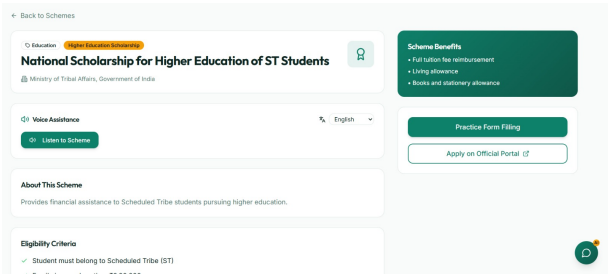


Figure 3.6: Scheme Detail Page Showing Eligibility, Benefits, and Required Documents

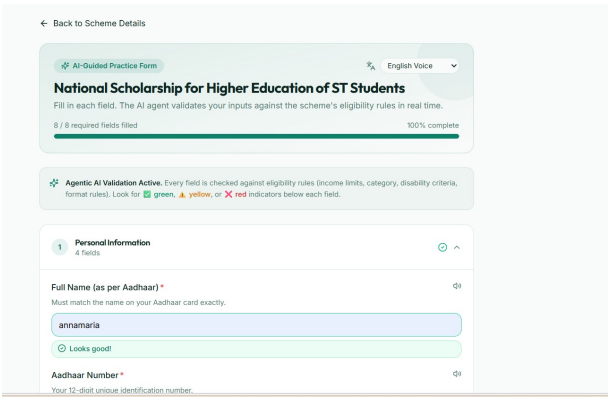


Figure 3.7: Guided Form-Filling Interface for Scheme Application

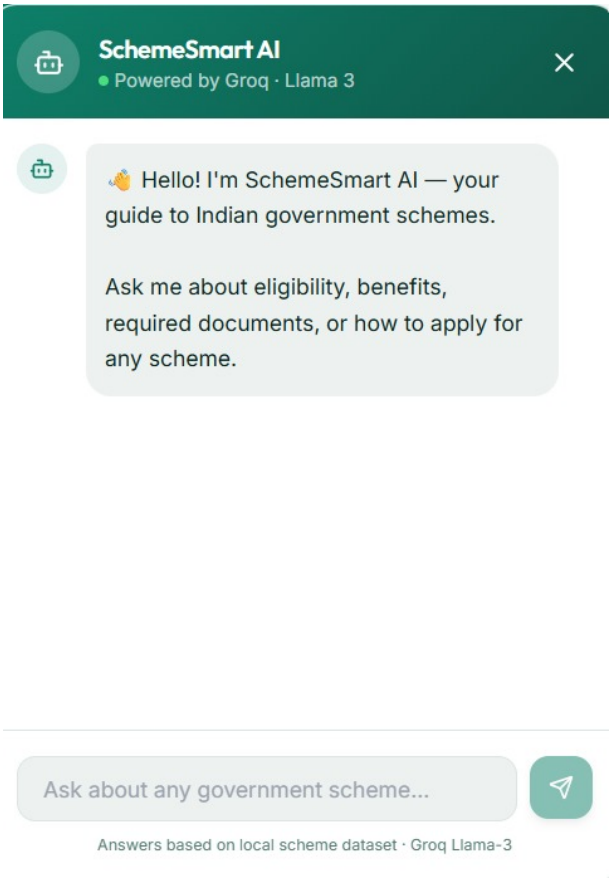


Figure 3.8: Chatbot Interface for Query Handling and User Assistance

placeins

3.6 Database Design

The SchemeSmart AI system uses Supabase, which is built on PostgreSQL, as the primary database for storing and managing application data. The database is designed to handle structured information efficiently, including user details, scheme data, and user interactions such as bookmarks and preferences. The schema consists of multiple tables such as users, schemes, bookmarks, and notifications, with appropriate relationships defined using primary and foreign keys to maintain data integrity. The users table stores authentication and profile information, while the schemes table contains details of government schemes including eligibility, benefits, and required documents. The bookmarks table allows users to save preferred schemes, and additional tables can be used for storing chat history or system logs.

Supabase was chosen as the database because it provides a robust and scalable Post-

greSQL backend along with built-in features such as authentication, real-time updates, and auto-generated APIs. Its support for SQL queries makes it highly suitable for handling structured data and performing complex filtering operations required in the recommendation system. Additionally, Supabase integrates seamlessly with FastAPI and modern frontend frameworks like React, making development and data management more efficient. The use of a cloud-based database also ensures high availability, security, and ease of maintenance, making it an ideal choice for the SchemeSmart AI system.

column_name	data_type	is_nullable
id	integer	NO
email	text	NO
password_hash	text	NO
created_at	timestamp with time zone	YES
instance_id	uuid	YES
id	uuid	NO
aud	character varying	YES
role	character varying	YES
email	character varying	YES
encrypted_password	character varying	YES

Table 3.1: User table schema (Supabase)

column_name	data_type	is_nullable
id	bigint	NO
scheme_id	text	NO
scheme_name	text	NO
sector	text	NO
description	text	YES
eligibility	jsonb	YES
benefits	jsonb	YES
documents_required	jsonb	YES
application_process	jsonb	YES
keywords	ARRAY	YES

Table 3.2: Scheme data table structure

column_name	data_type	is_nullable
id	integer	NO
user_id	integer	YES
full_name	character varying	YES
gender	character varying	YES
marital_status	character varying	YES
state	character varying	YES
district	character varying	YES
education	character varying	YES
occupation	character varying	YES
annual_income	character varying	YES

Table 3.3: User profile table schema

column_name	data_type	is_nullable
id	bigint	NO
user_id	bigint	YES
scheme_id	text	NO
scheme_name	text	NO
message	text	NO
is_read	boolean	YES
created_at	timestamp with time zone	YES

Table 3.4: Notification system table schema

3.7 Description of Implementation Strategies

The SchemeSmart AI system is implemented using a full-stack architecture that integrates a React-based frontend, a FastAPI backend, and Supabase as the database and authentication service. The implementation focuses on modularity, scalability, and efficient data handling to ensure a seamless user experience.

3.7.1 Backend Implementation using FastAPI

The backend is developed using FastAPI, a modern Python web framework known for its high performance and asynchronous capabilities. RESTful APIs are designed to handle functionalities such as user authentication, profile management, scheme recommendations, chatbot interaction, and form handling.

Asynchronous programming using `async/await` is employed to improve response time and efficiently manage multiple requests.

```
from fastapi import APIRouter


router = APIRouter()


@router.get("/recommend")
async def get_recommendations():
    data = await fetch_recommendations()
    return data
```

3.7.2 Database and Authentication using Supabase

Supabase is used as the backend-as-a-service platform, built on PostgreSQL. It provides a scalable and managed database along with built-in authentication features.

The system uses Supabase to:

- Store user profiles, scheme data, and form responses
- Manage user authentication (login and signup)

- Enable real-time data access

```
from supabase import create_client

url = "SUPABASE_URL"
key = "SUPABASE_KEY"
supabase = create_client(url, key)

response = supabase.table("schemes").select("*").execute()
schemes = response.data
```

3.7.3 Recommendation Engine Implementation

The recommendation engine is implemented using a rule-based filtering and scoring mechanism. User profile attributes are matched with scheme eligibility criteria, and a relevance score is calculated for each scheme.

```
def calculate_score(user, scheme):
    score = 0
    for condition in scheme["eligibility"]:
        if condition in user:
            score += 1
    return score
```

3.7.4 Chatbot Implementation using TF-IDF and RAG

The chatbot module is implemented using a hybrid Natural Language Processing approach combining TF-IDF (Term Frequency–Inverse Document Frequency) and Retrieval-Augmented Generation (RAG).

TF-IDF is used to convert textual data into numerical vectors, enabling similarity comparison between user queries and scheme documents. Cosine similarity is used to retrieve the most relevant documents. These documents are then passed to a language model through a RAG pipeline to generate contextual and accurate responses.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

query_vector = vectorizer.transform([user_query])
```

3.7.5 Guided Form-Filling Implementation

The system includes a guided form-filling feature to assist users in completing scheme application forms. Forms are defined using structured JSON schemas, and questions are dynamically rendered on the frontend.

User inputs are validated step-by-step, and feedback indicators (such as color-coded ticks) are used to assess the user's readiness for application.

```
form_schema = [
    {"id": "name", "question": "Enter your full name", "type": "text"},
    {"id": "income", "question": "Enter your annual income", "type": "number"}
]
```

3.7.6 Frontend Implementation using React

The frontend is developed using React, following a component-based architecture. This enables reusable UI components and efficient state management.

React Hooks such as `useState` and `useEffect` are used to manage application state and handle API calls.

```
useEffect(() => {
  fetch("/recommend")
    .then(res => res.json())
    .then(data => setSchemes(data));
}, []);
```

3.7.7 Authentication Strategy

Authentication is implemented using Supabase Auth, which provides secure user login and registration functionality. It supports token-based session management and integrates seamlessly with both frontend and backend components.

3.7.8 Evaluation Strategies

The system is evaluated using multiple testing approaches:

- Functional Testing to verify all features
- API Testing to ensure backend reliability
- User Testing to evaluate usability
- Performance Testing to measure response time

3.7.9 Summary

The implementation strategies adopted ensure that the system is robust, scalable, and user-friendly. The integration of Supabase, FastAPI, and React provides a strong foundation, while the use of TF-IDF and RAG enhances intelligent interaction through the chatbot. The guided form-filling feature further improves accessibility and reduces user errors.

3.8 Module Division

The SchemeSmart AI system is divided into multiple functional modules to ensure modularity, scalability, and efficient system design. Each module is responsible for a specific task and collectively contributes to the overall functionality of the system.

3.8.1 Module 1: User Profile Management

This module is responsible for collecting and managing user-specific information such as age, education, income, category, and location. It also stores user preferences and language details. The collected data serves as the primary input for the recommendation engine.

3.8.2 Module 2: Data Acquisition & Dataset Preparation

This module handles the collection and organization of scheme data. It stores curated government schemes (from sources such as MyScheme) along with eligibility criteria, benefits, and application links. It acts as the central data repository for the system.

3.8.3 Module 3: Eligibility Filtering Engine

This module applies rule-based eligibility checks on user profile data. It filters out schemes for which the user does not meet the criteria, ensuring that only relevant schemes are considered for further processing.

3.8.4 Module 4: AI-Based Scheme Ranking Module

This module ranks eligible schemes using Natural Language Processing techniques such as TF-IDF. It matches user needs with scheme descriptions and prioritizes the most relevant schemes.

3.8.5 Module 5: Scheme Recommendation Interface

This module is responsible for presenting the recommended schemes to the user in a clear and understandable format. It also provides explanations for why a scheme is recommended and includes links to official application portals.

3.8.6 Module 6: Application Form Template Module

This module defines the structure of application forms for different schemes. It contains form fields, instructions, and validation rules, which are used to guide users during the form-filling process.

3.8.7 Module 7: AI Voice-Assisted Form Guidance

This module provides step-by-step guidance to users while filling out application forms. It supports multiple Indian languages and helps users understand what information is required for each field.

3.8.8 Module 8: AI Chatbot Support Module

This module provides real-time assistance to users by answering scheme-related queries. It explains eligibility conditions, guides users through the platform, and assists in understanding form fields.

3.8.9 Module 9: Dashboard & Notification System

This module displays selected schemes and tracks user progress in the application process. It also sends notifications when new schemes are added or when updates are available.

3.8.10 Module 10: Backend & System Integration

This module integrates all system components using a backend framework. It manages API communication, data flow between modules, and ensures smooth coordination and scalability of the system.

3.8.11 Module Allocation to Team Members

The modules are distributed among project team members to ensure balanced workload and efficient development. Each member is responsible for the design, implementation, and testing of their assigned modules.

- Anna Maria Jose: Modules 3, 4, 9
- Anna Rose Shaji: Modules 6, 7
- Arunima V K: Modules 1, 10
- Ayisha Noureen: Modules 2, 5, 8

3.9 Work Schedule - Gantt Chart

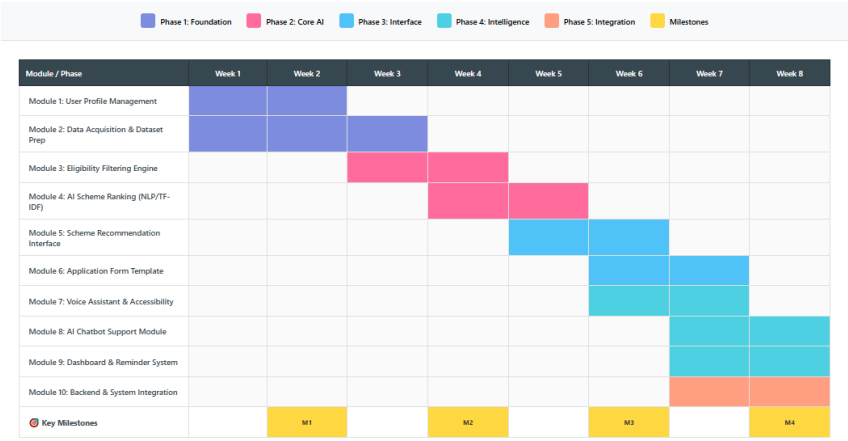


Figure 3.9: Gantt Chart Representing Project Timeline

Chapter 4

Results and Discussions

4.1 Overview

The SchemeSmart AI system effectively fulfills its primary objective of simplifying access to government welfare schemes by providing personalized recommendations based on user profiles. The system integrates multiple technologies, including a React-based frontend, a FastAPI backend, an AI-driven recommendation engine, and a Supabase database, to deliver a seamless and efficient user experience. Users can input their demographic and financial details, and the system processes this information to generate relevant scheme suggestions along with match scores. Additionally, the inclusion of a chatbot powered by a large language model enhances usability by allowing users to interact with the system in a natural and intuitive manner.

From a quantitative perspective, the recommendation engine demonstrates reliable performance in identifying suitable schemes using TF-IDF or semantic similarity techniques. The system efficiently analyzes user input against stored scheme data to produce accurate and meaningful results. Response times are optimized through effective API handling and lightweight data processing, ensuring that users receive recommendations and chatbot responses with minimal delay. The system is capable of handling multiple users simultaneously, highlighting its scalability and robustness when deployed in a real-world environment.

Further analysis shows that the system is highly flexible and can be extended to incorporate more advanced technologies such as embedding-based semantic search and improved ranking algorithms for enhanced accuracy. The modular architecture allows easy updates to individual components, such as upgrading the AI model or expanding

the scheme database. Overall, SchemeSmart AI proves to be a reliable, scalable, and user-friendly solution with strong potential for future enhancements, making it a valuable tool for increasing awareness and accessibility of government schemes.

4.2 Testing

The testing phase of the SchemeSmart AI system was conducted to ensure that all components of the application function correctly and efficiently. The system was tested in a step-by-step manner, covering all major functionalities including user authentication, profile input, scheme recommendation, and chatbot interaction. Initially, the user registration and login modules were tested to verify secure authentication and proper storage of user data in the Supabase database. Successful login and error handling for invalid credentials were also validated to ensure system reliability.

Following authentication testing, the profile input and recommendation modules were evaluated. Users were required to enter details such as age, income, and category, and the system was tested to confirm that these inputs were correctly processed by the backend. The recommendation engine was verified by checking whether relevant schemes were generated based on different user profiles. Multiple test cases with varying inputs were used to ensure that the TF-IDF or similarity-based ranking algorithm produced accurate and consistent results. Screenshots of the recommendation dashboard were recorded in chronological order to demonstrate the system's response under different conditions.

The chatbot functionality was also thoroughly tested by providing various user queries related to government schemes. The system successfully retrieved relevant information and generated meaningful responses using the language model. Additionally, overall system performance was evaluated by monitoring response times and ensuring smooth communication between frontend, backend, and database layers. The application was tested across different browsers and devices to confirm compatibility and usability. The results indicate that the system performs reliably, with all modules functioning as expected and delivering accurate and timely outputs.

Chapter 5

Conclusion

5.1 Conclusion

This project presented the design and development of SchemeSmart AI, an intelligent system aimed at simplifying access to government welfare schemes through personalized recommendations. By leveraging machine learning techniques such as TF-IDF–based ranking and user profile analysis, the system effectively matches users with relevant schemes based on their demographic and socio-economic details. The integration of a user-friendly interface, guided form-filling assistance, and a chatbot for real-time query handling enhances usability and ensures that even non-technical users can easily navigate and benefit from the platform.

Overall, the proposed system addresses the challenges of information overload, lack of awareness, and complexity in application procedures associated with government schemes. It demonstrates how AI-driven solutions can improve accessibility, transparency, and efficiency in public service delivery. Future enhancements may include multilingual support, integration with official government databases for real-time updates, and advanced recommendation models to further improve accuracy and user experience.

5.2 Future Scope

The proposed SchemeSmart AI system can be further enhanced by incorporating multilingual support to make it accessible to a wider range of users across different regions. Integration with official government databases and APIs can enable real-time updates of scheme information and eligibility criteria. Advanced machine learning models, such as deep learning–based recommendation systems, can be used to improve the accuracy and personalization of scheme suggestions. Additionally, a mobile application version and voice-based interaction features can be developed to increase usability and accessibility.

The system can also be expanded to include application tracking and notification services for a more comprehensive user experience.

Further improvements can include the integration of user feedback mechanisms to continuously refine recommendation accuracy based on user interactions. A dashboard for government authorities can be developed to analyze user trends and scheme effectiveness. The system can also incorporate document verification assistance using AI to guide users through the application process. Integration with digital identity systems can simplify authentication and eligibility verification. Moreover, implementing offline support and low-bandwidth optimization can make the system more accessible in rural areas. These enhancements will make the system more robust, scalable, and suitable for real-world deployment.

Bibliography

[1] F. Ricci, L. Rokach and B. Shapira, "Recommender Systems Handbook," Springer, 2015.

[2] G. Adomavicius and A. Tuzhilin, "Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art," IEEE Transactions on Knowledge and Data Engineering, vol. 17, no. 6, pp. 734–749, 2005, doi: 10.1109/TKDE.2005.99.

[3] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," Information Processing Management, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[4] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[5] S. Bird, E. Klein and E. Loper, Natural Language Processing with Python, O'Reilly Media, 2009.

[6] FastAPI, "FastAPI Documentation," Available: https://fastapi.tiangolo.com/, Accessed: 2026.

[7] Supabase, "Supabase Documentation: Open Source Firebase Alternative," Available: https://supabase.com/docs, Accessed: 2026.

[8] React, "React – A JavaScript Library for Building User Interfaces," Available: https://react.dev/, Accessed: 2026.

[9] P. Rajpurkar, R. Jia and P. Liang, "Know What You Don't Know: Unanswerable Questions for SQuAD," Proceedings of ACL, 2018.

[10] Ministry of Electronics and Information Technology (MeitY), Government of India, "Digital India Programme," Available: https://www.digitalindia.gov.in/, Accessed: 2026.

Appendix A: Presentation

FINAL PRESENTATION

SchemeSmartAI : AI-poweredGovernmentScheme

Recommendation System

Team Members
Anna Maria Jose
Arunima V K
Anna Rose Shaji
Ayisha Noureen

Project Guide:
Mr.Tonny Binoy

INTRODUCTION

- Government schemes provide financial and welfare support to citizens such as students, farmers, women, and small businesses.
- Information about these schemes is scattered across multiple portals, making discovery difficult.
- Many citizens are unaware of schemes they are eligible for.
- Understanding eligibility, documents, and application steps is often confusing.
- There is a need for a centralized and user-friendly system to simplify scheme access.

Table of Contents

1	Introduction
2	Problem Definition
3	Objectives
4	Scope and Relevance
5	System Design
7	Work Division – Gantt Chart
8	Software/Hardware Requirements
9	Conclusion
10	Future Enhancements
11	References
12	Results

OBJECTIVES

- ProvideAI-basedrecommendations for government schemes based on user eligibility.
- Simplify access to scheme details and application information
- Enable chatbot and voice assistance for user guidance.
- Offer practice form-filling support to reduce application errors.
- Provide a dashboard to organize and view scheme information.
- Provide a notification system to alert users when new matching schemes are added.
- Improve awareness and accessibility of government welfare schemes

PROBLEM DEFINITION

To develop an AI-powered web platform that recommends relevant Indian government welfare schemes based on a citizen's personal profile and eligibility, and provides comprehensive support through scheme details, multilingual voice assistance, an AI chatbot, guided practice form-filling, and a notification system — with the goal of simplifying access to welfare programs and improving awareness among citizens across all sectors.

Scope and Relevance

Scope

- AI-basedplatformto identifyandrecommend eligible government schemes for users.
- Integration of chatbot and voice assistance to simplify scheme discovery and guidance.
- Supports guided form-filling practice to help users apply correctly.

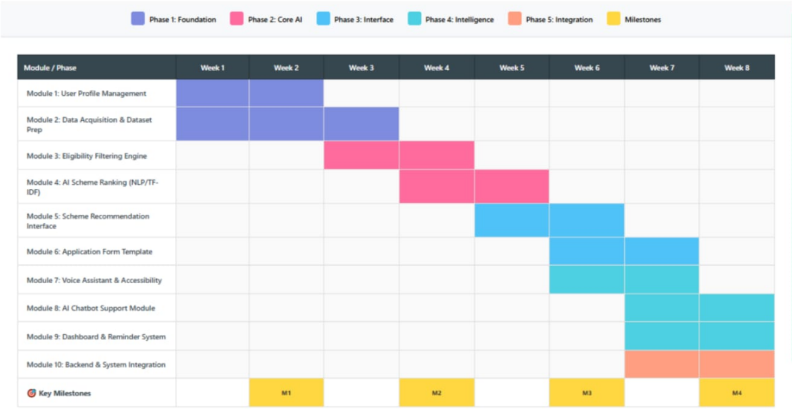
Project Coverage

- Profile-based eligibility filtering and scheme recommendation.
- Detailed scheme information including eligibility, documents, and application steps.
- AI chatbot and voice-guided assistance for user support

Relevance

- Addresses the problem of low awareness of government welfare schemes.
- Simplifies access to scattered scheme information across multiple portals.
- Helps citizens identify and apply for schemes they are eligible for.

WORK DIVISION



MODULES

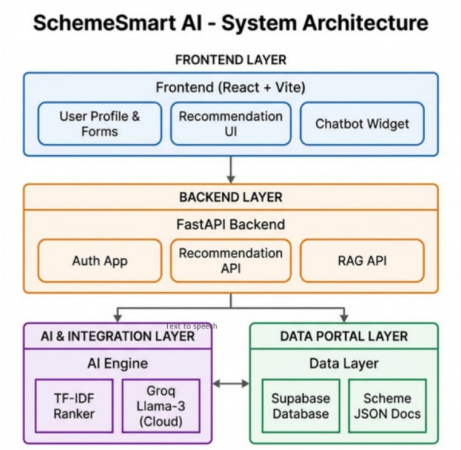
Module 1: User Profile Management

- Collects age, education, income, category, and location
- Stores user language and preference details
- Provides input for filtering and ranking

Module 2: Data Acquisition & Dataset Preparation

- Stores curated education schemes from MyScheme
- Maintains scheme eligibility rules, benefits, and application links
- Acts as the central data source for recommendation logic

SYSTEM DESIGN



MODULES

Module3:EligibilityFiltering Engine

- Applies rule-based eligibility checksonuserprofile data
- Filters out schemes the user is not eligible for
- Ensures onlyrelevanteducationschemes moveforward

Module 4: AI-Based Scheme Ranking Module

- Uses NLP (TF-IDF) to rank eligible schemes
- Matches user needs with scheme descriptions
- Prioritizes the most suitableschemesfortheuser

Module 5: Scheme Recommendation Interface

- Displays top-rankedschemes inasimple, clearformat
- Explains why each scheme is recommended
- Provides official government application links

ALGORITHM

Retrieval-AugmentedGeneration (RAG) for Scheme Chatbot

- Input: User query, full scheme knowledge base (JSON files + Supabase)
- Output: Context-grounded chatbot response

Steps:

1. Vectorise the user query using TF-IDF.
2. Compute cosine similarity against all scheme documents.
3. Retrieve the top-5 most relevant schemes (score > 0.01 threshold).
4. Format retrieved schemes as structured context text.
5. Inject context into the system prompt for Groq Llama-3.3-70B.

Generate and return a concise, scheme-specific response.

ALGORITHM

Eligibility Filtering Algorithm

- Input: User profile data, scheme eligibility rules
- Output: Filtered and validated scheme list

Steps:

1. Extract user demographic fields (age, income, category, state, sector).
2. For each scheme, check if its sector matches the user's selected sectors.
3. Apply a ×2.0 boost to matching sectors; ×0.25 penalty to non-matching sectors.
4. In practice forms, validate each field entry against the scheme's eligibility rules.
5. Return Pass / Fail / Warning status for each eligibility criterion.

MODULES

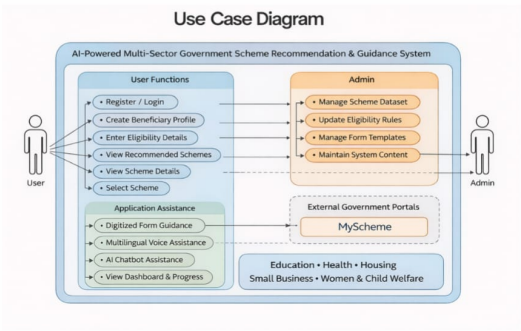
Module 9: Dashboard & Notificaton System

- Displays selected schemes and guidance progress
- Notifies user whenever a scheme is added
- Helps users keep track of application steps

Module 10: Backend & System Integration

- Integrates all modules using a Python–Flask backend
- Handles data flow, APIs, and logic coordination
- Ensures scalability and smooth system performance

DESIGN MODEL



ALGORITHM Scheme Ranking Algorithm (TF-IDF)

Input:User profile,schemedocuments
Output: Top 12 ranked schemes

Steps:

1. Convert schemes into weighted TF-IDF documents (name, sector, description, eligibility).
2. Generate a query vector from user profile details.
3. Compute cosine similarity between user query and scheme vectors.
4. Apply bonus scores for demographic matches.
5. Apply sector boost or penalty.
6. Rank and return the top matching schemes.

CONCLUSION

- AI-based platform tohelpusers discover relevant government schemes based on eligibility.
- Uses rule-based filtering and TF-IDF retrieval for accurate scheme recommendations and details.
- Integrates TF-IDF + RAG chatbot, voice assistance, and form guidance for easier understanding.
- Improves accessibility and awareness of welfare schemes and can be extended to web/mobile platforms.

FUTURE ENHANCEMENTS

- **Expansion to More Government Sectors**

Extend the platform to include additional welfare sectors such as agriculture, employment, and senior citizen schemes for wider public benefit.

- **Integration with Official Government Portals**

Enable direct application submission and real-time status tracking through integration with official portals like MyScheme and other e-governance platforms.

- **Mobile Application with Regional Language Support**

Develop a mobile app with enhanced multilingual voice assistance to improve accessibility for rural and non-technical users.

SOFTWARE/HARDWARE REQUIREMENTS

HARDWARE REQUIREMENTS

- Devices: Desktop / Laptop / Android & iOS smartphones
- Processor & Memory: Intel i5 (or equivalent), 8 GB RAM
- Storage & Network: 256 GB SSD, stable Internet connection

SOFTWARE REQUIREMENTS

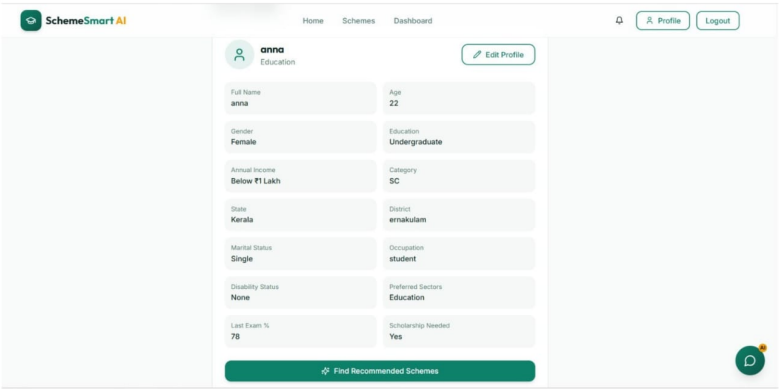
- Frontend: React 18 + TypeScript (Vite build tool)
- Styling: Tailwind CSS + shadcn/ui components
- Backend: Python 3.11 + FastAPI + Uvicorn
- Database & Tools: Supabase, Git/GitHub, VS Code

RESULTS

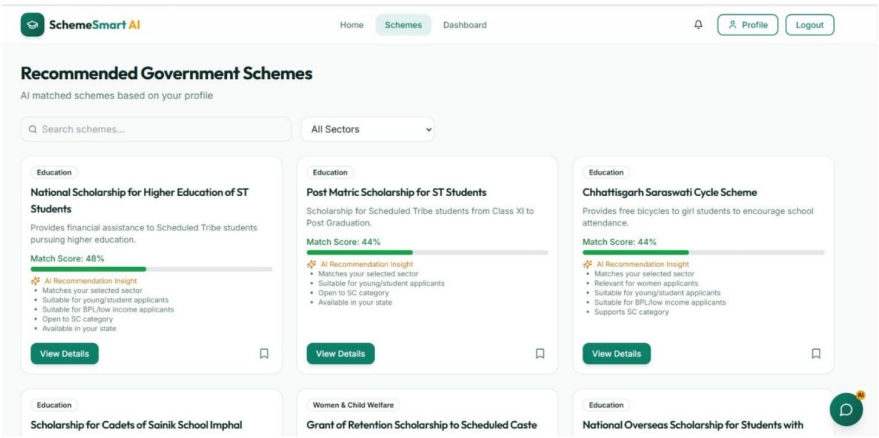
REFERENCES

- FastAPI Documentation – FastAPI official guide for building backend APIs. https://fastapi.tiangolo.com
- Python 3 Documentation – Official Python documentation for syntax, libraries, and best practices. https://docs.python.org/3/
- React.js Documentation – Official React.js documentation for building frontend components. https://reactjs.org/docs/getting-started.html
- Git & GitHub Documentation – Version control system for code collaboration and management. https://docs.github.com/en
- OpenAI / NLP Libraries – For natural language processing, chatbot, and AI integration. https://huggingface.co/docs/transformers/index

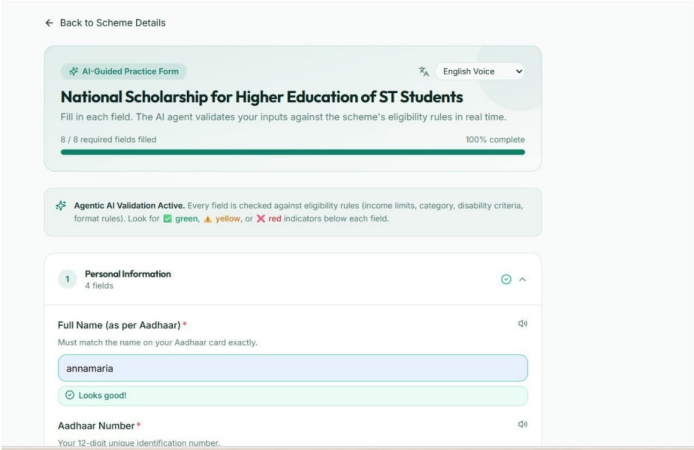
RESULTS



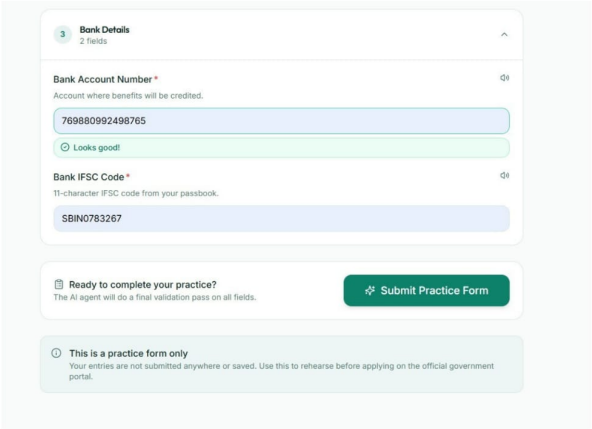
RESULTS



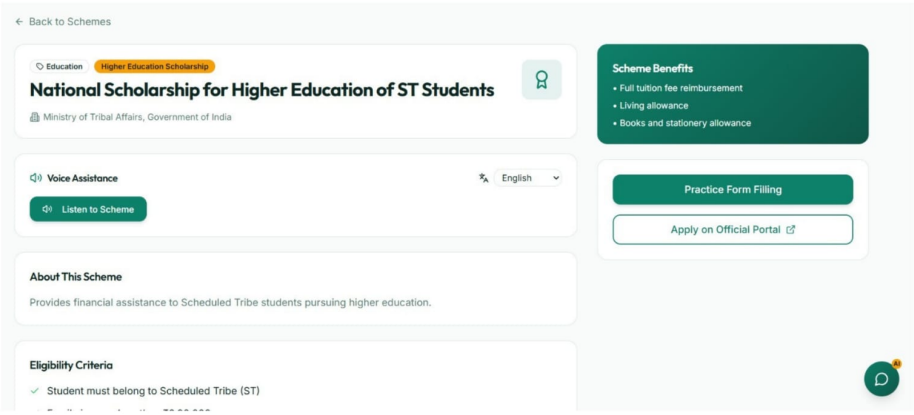
RESULTS



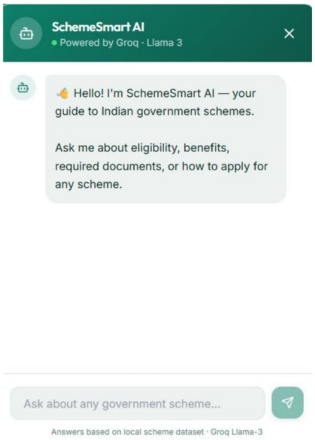
RESULTS



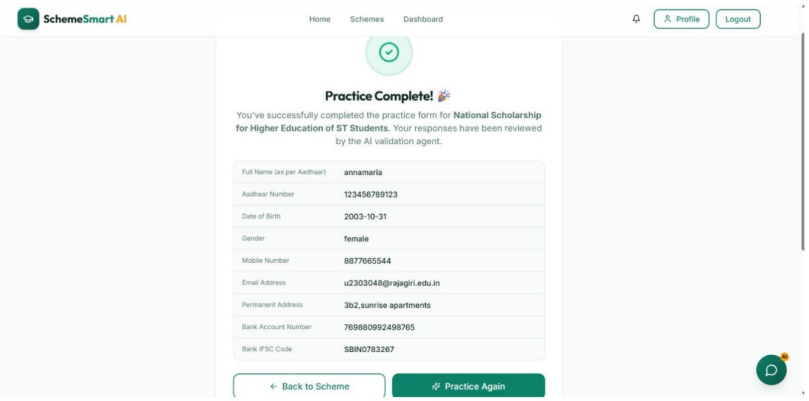
RESULTS



RESULTS



RESULTS



THE END